DEVELOPER OFFICE HOURS

Building a GitHub Code Review Squad

Three agents that write code, push it, review it, remember what they found, and email the report. Built live in one hour, from an empty file.

Ruth Mutua & Abdullahi · Prescott Data

JarvisCore 1.3.0 · 27 August 2026

Agenda

01	**The problem**	Where agents break on the way to production, and what we build to fix it
02	**JarvisCore basics**	Two profiles, one mesh, and the choice between a prompt and code
03	**The build**	What you need, how the credentials stay out of reach, and the four steps
04	**Close**	Where the code lives, and where to keep asking

01

The problem

Where agents break on the way to production, and what we build to fix it

4 Everyone's agent works in the notebook and dies on the way out.

5 Three agents, one repo, two services.

Everyone's agent works in the notebook and dies on the way out.

WHERE THEY BREAK

01 **Credentials.** The token ends up in the prompt, the log, or the repo.

02 **Memory.** Every run is the first run. Nothing carries over.

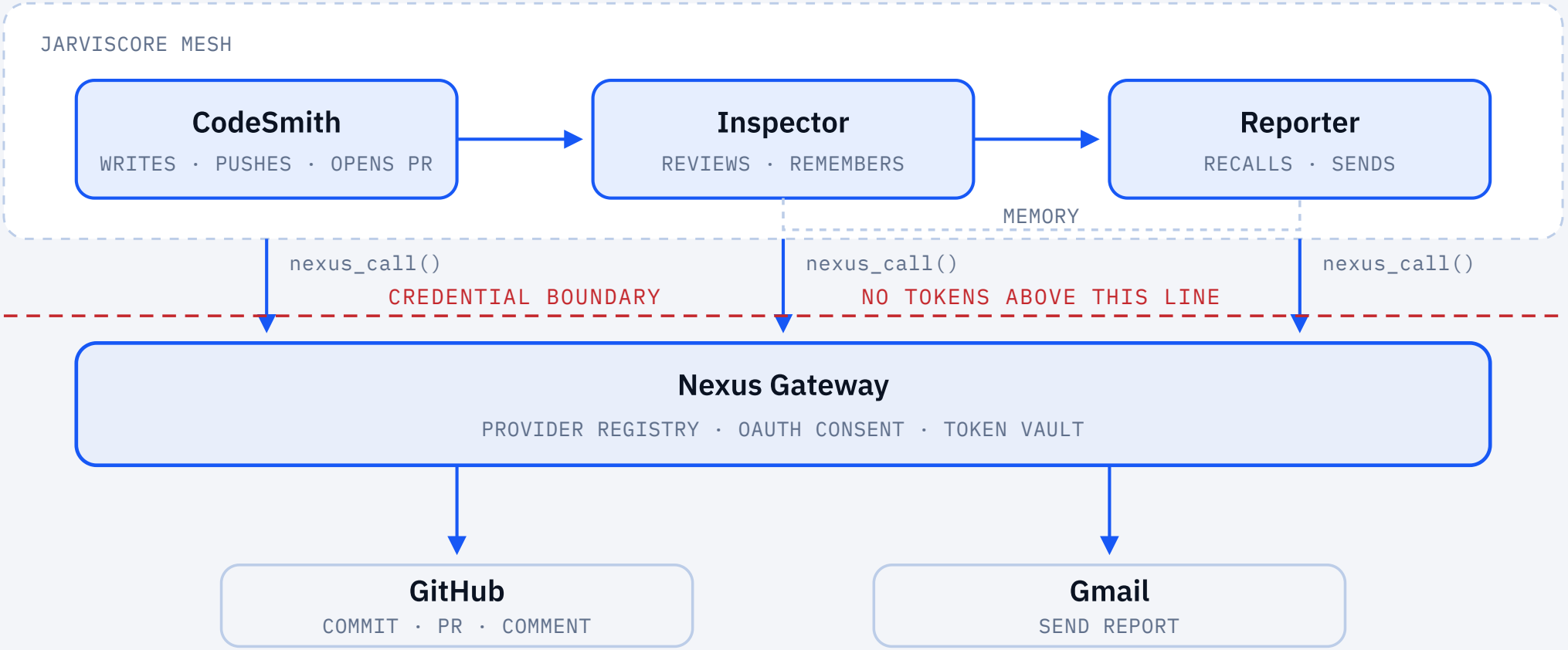
03 **Reaching the real world.** The demo reads a file; production needs GitHub, Gmail, Slack, Jira.

WHAT WE'LL SHOW

All three, fixed in front of you, in four steps you can run yourself afterwards, and every call the agents make printed as it happens, so you can see exactly where the work is going.

If your agent is broken in one of those three ways, that's the session. Bring it to the chat.

Three agents, one repo, two services.



— Every external call goes through the Nexus gateway. Nothing above the red line holds a token.

02

JarvisCore basics

Two profiles, one mesh, and the choice between a prompt and code

```
7  Most frameworks hand the model a menu. JarvisCore lets it write the tool.
8  Two profiles. One mesh.
9  An AutoAgent is three attributes.
10 A CustomAgent is one method.
11 The mesh is four lines.
12 Judgement in a prompt. Mechanics in code.
```

Most frameworks hand the model a menu. JarvisCore lets it write the tool.

```
Most frameworks:   you define tools    →  model picks one     →  the tool runs
JarvisCore:        you describe a goal →  model writes Python →  a sandbox runs it
```

And when you'd rather not leave something to a model, you write that part yourself. That choice is the whole API surface.

150
SERVICE BUNDLES

1,227
INTEGRATION ATOMS

4
MEMORY TIERS

2
PROFILES TO LEARN

Two profiles. One mesh.

PROFILE	YOU WRITE	THE FRAMEWORK WRITES	REACH FOR IT WHEN
CustomAgent	`execute_task()`: the logic	Mesh routing, discovery, lifecycle, infrastructure injection	The answer is deterministic. Don't pay a model to do arithmetic.
AutoAgent	`role`, `capabilities`, `system_prompt`	Look-decide-act loop, planning, tool generation, sandboxed execution, self-repair	The work needs judgement: reviewing, writing, deciding.

— Our squad uses both. That's not a demo contrivance, it's the actual rule: **deterministic work goes in code, judgement goes in a prompt.**

An AutoAgent is three attributes.

```
class Inspector(AutoAgent):
    role          = "inspector"
    capabilities  = ["code_analysis", "security_analysis"]
    system_prompt = """You review code that is already on GitHub..."""
```

No tool definitions. No JSON schema. No executor wiring. The kernel plans the steps, writes the Python, runs it in a sandbox, and repairs it when it fails.

WHAT THOSE THREE DO

role The name other agents route to.

caps What it advertises to the mesh, so work finds it.

prompt The job description. This is where your thinking goes.

A CustomAgent is one method.

```
class WarehouseAgent(CustomAgent):
    role         = "warehouse"
    capabilities = ["data_access"]

    async def execute_task(self, task):
        # you write this. no model involved, no tokens spent.
        return {"status": "success", "output": {...}}
```

Same mesh, same routing, same infrastructure injection: you just supply the body. **Use it wherever a model would only add latency and risk.**

The mesh is four lines.

```
mesh = Mesh(config={"nexus_gateway_url": ..., "redis_url": ...})
mesh.add(CodeSmith)
mesh.add(Inspector)
mesh.add(Reporter)
await mesh.start()

results = await mesh.workflow("review", [
    {"agent": "codesmith", "task": "Write it and push it"},
    {"agent": "inspector", "task": "Review it", "depends_on": [0]},
    {"agent": "reporter",  "task": "Email it",  "depends_on": [1]},
])
```

`depends_on` is the whole orchestration language. Steps with no dependency between them run in parallel; the engine handles claiming, retries and crash recovery.

Judgement in a prompt. Mechanics in code.

THE THREE AGENTS DECIDE THINGS

1. **CodeSmith**: what does plausible-but-flawed payments code look like?
2. **Inspector**: which of these are real defects, and how bad?
3. **Reporter**: what does someone actually need to read?

SIX CALLS STAY ORDINARY PYTHON

```
create_branch()
commit_file()
read_file()
open_pr()
comment_on_pr()
send_email()
```

Deterministic, so a model adds nothing but risk, and printing every one of them is what makes the run followable.

— Same credential guarantee either way: every call goes through `proxy.call(connection_id, …)`. **Ask which half a job belongs in before you write it.** That one question is most of the design.

03

The build

What you need, how the credentials stay out of reach, and the four steps

14 What you need, and what you don't.

15 Your agent never holds the token.

16 Four steps. Each runs on its own.

What you need, and what you don't.

NEED

- ✓ Python 3.10+
- ✓ `pip install jarviscore-framework[redis]`
- ✓ Docker, for Redis: one compose command
- ✓ **One** LLM credential, from whoever you like. JarvisCore is model-agnostic: Azure OpenAI, Anthropic, Gemini, or any OpenAI-compatible endpoint
- ✓ A GitHub account

DON'T NEED

- – A **second** model provider: one credential covers the whole demo
- – A Google Cloud project: the gateway already has a client on `google-workspace`
- – Any vector database
- – A Nexus deployment: we used ours on the day
- – To have written an agent before

```
git clone https://github.com/Prescott-Data/office-hour-demos
cd office-hour-demos/2026-08-27-oh-github-code-review-squad && pip install -r requirements.txt
```

Your agent never holds the token.

WHAT THE AGENT WRITES

```
async def main():
    r = await nexus_call("PUT", url,
            json={"content": b64})
    return {"success": r["ok"]}
```

One door. No headers, no secrets, nothing to leak.

WHAT NEXUS DOES

Resolves an opaque `connection_id` to a live credential, applies it to the request, and returns only the response.

Nexus is also where the provider gets registered in the first place: catalog, OAuth consent, token refresh, revocation.

— Proof, live: after a run we grep the generated code for `token|secret|bearer` and get nothing back.

Four steps. Each runs on its own.

RUN	WHAT HAPPENS	THE MOMENT
`write_code.py`	Writes the module, prints it, checks the defects	Three attributes, zero infrastructure
`push_and_open_pr.py`	Consent → branch → commit → pull request	Every call on screen, no token anywhere
`review_pr.py`	Reads it **back from GitHub**, reviews, comments	Run twice; the second run remembers
`email_report.py`	Drafts the report, then **asks before sending**	The inbox opens on screen

04

Close

Where the code lives, and where to keep asking

18 Everything we typed is in the repo.

Everything we typed is in the repo.

So, what's broken in yours?

CODE	github.com/Prescott-Data/office-hour-demos
DOCS	jarviscore.developers.prescottdata.io
KEEP ASKING	#jarviscore on Discord